

Turning the demo into a multi-client platform

The seven areas to agree with the developer, the fifteen questions to get answered, and the order the work must happen in. Includes what is already built and what is still only a screen.

READ THIS FIRST

Roughly half of the foundation described here now exists in the repository — the multi-tenant database, the security model, the event backbone and the schema for all seven areas. **None of it is deployed.** Three database migrations and two server functions are written, tested and committed, and the live site still runs code that accepts “notanemail” as a valid email address.

Ten minutes of deployment separates a good demo from a working product. Do that before commissioning anything new.

Where the build actually stands

AREA	STATE	DETAIL
Multi-client database	BUILT	26 of the 28 tables the specification requires. Every table carries a tenant id.
Tenant isolation (RLS)	BUILT	Enforced in the database on all 34 tables, not in the browser. Never tested with two live accounts.
Staff roles	BUILT	Owner, admin, manager, staff, read-only — enforced in policy, not by hiding buttons.
Multiple locations & assistants	BUILT	Schema supports both. No interface yet.
Event backbone	BUILT	Durable outbox with retry, backoff and dead-lettering. A CRM outage delays a sync; it cannot lose one.
Assistant Studio	SCHEMA ONLY	Versioning and immutability enforced by the database. No interface, no website scanner.
Phone routing	SCHEMA ONLY	All six routing modes modelled. No carrier account, so no route has ever carried a call.
CRM integrations	PARTIAL	HubSpot written end to end. Never authorised — no credentials exist.
Email connection	NOT BUILT	Only outbound alerts. No Google or Microsoft sign-in.
Analytics	SCHEMA ONLY	Fact table stores numerator, denominator and completeness. Nothing computes it yet.
Niche packs	SCHEMA ONLY	Table exists. No packs written.
Live phone answering	NOT LIVE	Worker built and logic tested; never run against a real call.

1 • Proper multi-client architecture

“This cannot be built as one dashboard with different demo data. Every customer needs their own isolated workspace, users, assistants, numbers, leads, integrations and analytics.”

This one is largely done. Confirm each point rather than commission it:

MUST BE TRUE	STATE	EVIDENCE
Supabase is the source of truth	YES	All records live in Postgres. n8n holds no state.
Every table has a tenant id	YES	Called workspace_id. Same concept, different word.
Row Level Security blocks cross-tenant reads	YES	On every table, resolved through a membership function the client cannot forge.
Owner / admin / team-member roles	YES	Five roles. Read-only genuinely cannot write.
One client, multiple locations	YES	Locations table with per-site hours, numbers and routing.
One client, multiple assistants	YES	Assistants belong to a workspace and optionally to a location.

THE ONE THING STILL UNPROVEN

Isolation is enforced but has never been **tested** with two real accounts. The test is written and takes one command. Create a second account and it can be proven the same day — until then it is a design, not a fact.

2 • The AI Assistant Studio

Clients must build and control their own receptionist. Today the assistant is hard-coded — changing it means a developer editing a file. This is the largest piece of unbuilt work.

“The website scanner must not immediately push information live. The customer must review and approve the knowledge before the assistant uses it.”

Already enforced by the database

- **A published version can never be edited.** A trigger refuses it. Rollback means publishing an earlier version — so what the assistant said last Tuesday stays provable.
- **Exactly one published version per assistant.** Two would make “what is my receptionist currently saying?” unanswerable.
- **Scanned knowledge starts unapproved.** The safe state is the default state. Prices, legal, medical and emergency content are flagged for a human even when the AI is confident.
- Every extracted fact keeps its source page, so a customer can ask “where did that come from?” and get an answer.

Still to build

Website scanner, review screen, voice picker with preview, question builder, test simulator, and the publish/rollback interface. Everything the customer actually touches.

3 • Real phone connection and routing

“I understand we cannot take control of a number just by sending an SMS code. The code can verify ownership, but the customer must still use forwarding, SIP, a managed number or number porting.”

This is correct and worth being firm about. A verification code proves someone can receive a message. It does not make us the carrier and it cannot redirect a call. Any supplier who says otherwise is either confused or selling.

The six modes, all modelled in the database

MODE	WHAT IT NEEDS
AI answers every call	Unconditional forwarding, or a managed/ported number
AI answers missed calls only	No-answer forwarding from the carrier
AI answers when busy	Busy forwarding
AI answers outside hours	Schedule at the carrier, or a route we control
AI calls back after a missed call	Only works if we received the original call event
AI on a custom schedule	Route controlled by us

THE DATABASE NOW REFUSES A FALSE CLAIM

A phone route **cannot** be marked active without a stored, verified inbound test call. This is not a rule in the interface that a future change might forget — the database rejects the write. “Active” means a real call reached the right assistant, and nothing else.

Recommended first version

- Alsaiti holds the carrier account and provisions a hidden destination number.
- The customer keeps advertising their existing number and sets up forwarding.
- We generate the exact instructions for their carrier.
- A real test call runs before anything says live.
- Porting stays optional, for customers who want full control later.

Ask him directly

- Which voice platform — Retell, Vapi, LiveKit or custom? *A LiveKit worker exists but has never handled a real call. Retell is faster to launch; keep the layer swappable either way.*
- Which carrier — Telnyx, Twilio or another? *Telnyx code is written and deployed already.*
- How are inbound call webhooks secured? *Signature verification is already implemented.*
- How are calls recorded, transcribed, and consent announced per country?
- What happens when the AI or the provider is unavailable? *Configure the fallback before the first real call, not after the first outage.*
- Can calls transfer to a human, and what happens when nobody answers?

4 • n8n for integrations — Supabase stays the backend

“I want n8n to act as the automation and integration layer, not as the database or the core backend.”

Right principle, and the mechanism for it is already built. The flow is:

```
call / form / chat / email
-> Alsaiti saves the lead in Supabase
-> an event is written in the SAME transaction
-> a consumer delivers it to the CRM
-> success, retry or dead-letter is recorded
```

The lead and its event commit together or not at all. That is what guarantees the promise below — it is not a matter of careful coding.

“Even when HubSpot, Salesforce or another CRM is offline, the lead must remain safe in Supabase and sync later.”

Already built

- Durable queue with retry, exponential backoff and a dead-letter state.
- Duplicate prevention — the same event cannot be delivered twice however often a provider retries.
- Per-client credentials, encrypted before they touch the database.
- Field-mapping table, versioned per connector.
- Health states: draft, testing, active, degraded, action required, paused, disconnected.

My recommendation: do not add n8n yet

The queue already does the hard part. One consumer process covers the first customers. Adding n8n before a second connector exists buys a server to maintain, a licensing question about embedding its editor, and no new capability. The design allows it to be added later without rework — so add it when the connector list justifies it, not before.

First connectors, in order of value

HubSpot (written), Google Calendar, Pipedrive, Google Sheets, generic webhook, generic REST. Salesforce, GoHighLevel and Dynamics after the first pilots. Resist advertising “any backend” until several are genuinely proven.

5 • Email connection through sign-in, not passwords

“The application must clearly distinguish between an email being verified and a mailbox being fully connected.”

Not built. Today the platform only sends its own alerts. Customers cannot connect their mailbox at all.

THE DATABASE ALREADY DEFINES WHAT “CONNECTED” MEANS

A mailbox is only Active when **three separate things** have succeeded: the token refreshes, a test message actually sent, and the reply subscription is live. Each is stored as its own fact, so a half-working connection cannot round itself up to “connected”.

The experience to build

- Click **Connect Google** or **Connect Microsoft** — never type a password.
- Approve the permissions, send a test email, confirm replies are detected.
- See the connected mailbox address and its health.
- Disconnect and delete the stored access at any time.

Ask him to cover

Token refresh and re-authorisation before expiry; shared mailboxes; templates and signatures; delivery and reply logs. One warning worth passing on: **mailbox subscriptions expire in days, not months** — Microsoft around three, Google seven. Renewal has to be a scheduled job with a visible health state, because the failure mode is silence, and nobody notices silence.

6 • Better leads and analytics

“I want the analytics to demonstrate business return, not just show attractive charts.”

The lead record now has columns for sentiment, AI confidence, estimated and actual value, outcome reason, appointment link, consent history and a full interaction timeline. Recording and transcript attach to the call. **The screens that show all this do not exist yet.**

Two design decisions worth insisting on

- **The lead score must be explainable.** Every component is stored separately — intent, fit, urgency, contactability, engagement, value, risk — with the model version. A customer who asks why a lead scored 72 gets an answer instead of a shrug.
- **Every metric stores its denominator and how complete the data was.** A conversion rate without its denominator cannot be audited, and a rate calculated from half the data without saying so is a lie with a decimal point in it.

Everything on your analytics list is supported by that fact table — calls, answer and miss rates, AI versus human, transfers, bookings, qualification, conversion, attributed revenue, response times, drop-off, top enquiry reasons, performance by hour, source and location, and integration success rates.

7 • Niche templates, not separate applications

“We are not building a different application for every niche. We are building one platform where the configuration, knowledge, workflows and lead fields change.”

The table for this exists. A pack holds the vocabulary, services, qualification questions, intents, required disclaimers and scoring weights for an industry. A customer adopts a pack and may override any part of it — so nine industries share one codebase.

DO NOT WRITE NINE PACKS FIRST

Write **two**, for the industries you already have customers in, and prove the mechanism end to end. Seven speculative packs is seven guesses about businesses nobody has sold to yet, and each one has to be maintained.

The fifteen questions — and the answers I can already give

Several have factual answers from the existing code. Marked **ANSWERED** below; the rest are genuine decisions for the call.

#	QUESTION	POSITION
1	Proposed production architecture?	ANSWERED. Static site on Cloudflare, Supabase for data and auth, server functions for provider work, durable outbox for hand-off.
2	What is still demo or hard-coded?	ANSWERED. Assistant behaviour, analytics numbers, niche content, and every integration except HubSpot.
3	How is tenant isolation implemented?	ANSWERED. Database-level, on every table. Needs proving with two live accounts.
4	Voice provider and carrier?	Open. Telnyx code exists; voice runtime undecided.
5	First real telephone connection method?	Recommend managed number plus guided forwarding.
6	Missed-call detection and callback?	Only works for calls that reached our route. Be clear about that limit.
7	Google and Microsoft email sign-in?	Not started. Scopes and renewal are the real work.
8	How do workflows scale to hundreds of clients?	ANSWERED. Configuration per tenant, one shared consumer. Never clone a workflow per customer.
9	Which CRMs in release one?	Recommend HubSpot and Google Calendar.
10	What is genuinely possible in version one?	See the phase table below.
11	What comes later?	Website scanner, niche packs, advanced analytics, SIP.
12	Infrastructure and per-minute call costs?	Usage ledger is built so this is measured , not estimated. Needs provider pricing.
13	Failures, retries and outages?	ANSWERED for data. Carrier-level fallback still needs configuring.
14	What testing before a client is shown live?	ANSWERED. A real inbound test call, stored. The database enforces it.
15	What accounts and credentials does he need from you?	See the final page.

The order the work must happen in

“Do not start by adding more visual dashboard features while the telephony, data isolation and core lead system remain simulated.”

PHASE	STATE	WHAT REMAINS
1. Multi-tenant database, auth, security	MOSTLY DONE	Deploy it. Prove isolation with two accounts.
2. Real leads, detail pages, timelines	PART DONE	Data model complete; the rich lead screen is not built.
3. Assistant Studio and website knowledge	SCHEMA ONLY	Largest single piece. 3–4 weeks.
4. Real telephony and routing	BLOCKED	Needs a funded carrier account before work can start.
5. Email sign-in connection	NOT STARTED	Needs Google and Microsoft applications registered.
6. Integration engine and CRM connectors	PART DONE	Queue built. HubSpot needs credentials to prove.
7. Analytics and revenue reporting	SCHEMA ONLY	Agree the definitions before building charts.
8. Billing, onboarding, monitoring	NOT STARTED	Health endpoint exists; nothing watches it.

Realistic total: 13–20 weeks. The variance is dominated by provider access rather than by engineering — phases 4 and 5 cannot begin without accounts, and that has already held up two integrations for weeks.

What he needs from you — this is the bottleneck

WHAT	FOR	BLOCKS
Deploy the database and functions	Everything already written	All of it
Verified email sending domain	Lead alerts reaching anyone but you	Every notification
A second test account	Proving tenant isolation	Any pilot customer
Funded carrier account	Real phone routing	Phase 4 entirely
Google & Microsoft app registrations	Email sign-in	Phase 5 entirely
CRM developer credentials	Proving HubSpot	Phase 6 sign-off
Decision: recording on or off, and retention	Legal position before launch	Phase 3 schema
Hosting budget for the voice worker	Live call answering	Phase 4 go-live

THE ONE INSTRUCTION THAT MATTERS MOST

Do not let the next piece of work be another screen. The database, security model, event backbone and schema for all seven areas are built and waiting. The gap between a convincing demo and a working product right now is **one deployment command** — not another dashboard.

Never send an API key, password, access token or recovery code through chat, email or a screenshot — including to a developer. Enter them directly into the provider dashboard.